

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №6
по дисциплине «Разработка кроссплатформенных приложений»
на тему «Сетевое взаимодействие в Java»

Выполнили: ст. гр. 23ВВИ1

Николаев А.В.

Куликовский В.А.

Приняли:

доцент к.т.н. Юрова О.В.

доцент к.т.н. Карамышева Н.С.

Пенза 2026

Цель работы: научиться создавать клиент-серверные приложения с использованием стандартных классов Java.

Задание: модифицировать приложение из предыдущей лабораторной работы, реализовав клиент-серверную архитектуру, обеспечивающую распределенное вычисление определенного интеграла на нескольких вычислительных узлах (клиентах) при этом каждый узел использует несколько нитей, как в предыдущей работе. Сервер не занимается вычислениями, а лишь реализует взаимодействие с пользователем и агрегацию результатов вычислений от клиентов. Нечетные варианты используют протокол UDP, а четные TCP. Оформление лабораторной работы должно быть выполнено в соответствии с требованиями, приведенными в Приложении 2.

Ход работы

При запуске приложения запускается UDP сервер на порту 5000.

```
private void startComputeServer() {
    try {
        computeServer = new
ComputeServer(ComputeServer.DEFAULT_PORT);
        setTitle("RecIntegral UDP Server - port " +
computeServer.getPort());
    } catch (SocketException e) {
        JOptionPane.showMessageDialog(this, e.getMessage(), "UDP
Server Error", ERROR_MESSAGE);
    }
}
```

Сервер разбивает на подзадачи для каждого клиента, ожидает их выполнения и объединяет результаты их работы.

```
public double calculate(double lowLimit, double highLimit, double
step) throws Exception {
    if (clients.isEmpty()) {
        throw new IllegalStateException("No compute clients
registered on UDP port " + getPort());
    }

    String taskGroupId = UUID.randomUUID().toString();
    List<ClientInfo> taskClients = new ArrayList<>(clients);
    List<CompletableFuture<Double>> futures = new ArrayList<>();

    double length = (highLimit - lowLimit) / taskClients.size();
    try {
```

```

        for (int i = 0; i < taskClients.size(); i++) {
            double partLow = lowLimit + i * length;
            double partHigh = (i == taskClients.size() - 1) ?
highLimit : lowLimit + (i + 1) * length;
            String taskId = taskGroupId + "-" + i;
            CompletableFuture<Double> future = new
CompletableFuture<>();
            pendingResults.put(taskId, future);
            futures.add(future);
            send(taskClients.get(i), "TASK;" + taskId + ";" +
partLow + ";" + partHigh + ";" + step);
        }

        double result = 0.0;
        for (CompletableFuture<Double> future : futures) {
            result += future.get(CLIENT_TIMEOUT_SECONDS,
TimeUnit.SECONDS);
        }
        return result;
    }
}

```

Сервер слушает порты, закрепленные за клиентами и при получении сообщения обрабатывает их.

```

private void handleMessage(InetAddress address, int port, String
message) {
    String[] parts = message.split(";", 3);
    if ("REGISTER".equals(parts[0])) {
        registerClient(address, port);
        return;
    }

    if ("RESULT".equals(parts[0]) && parts.length == 3) {
        CompletableFuture<Double> future =
pendingResults.remove(parts[1]);
        if (future != null) {
            future.complete(Double.parseDouble(parts[2]));
        }
        return;
    }

    if ("ERROR".equals(parts[0]) && parts.length == 3) {
        CompletableFuture<Double> future =
pendingResults.remove(parts[1]);
        if (future != null) {
            future.completeExceptionally(new
IllegalStateException(parts[2]));
        }
    }
}

```

Клиент уведомляет сервер о своем существовании.

```

public static void main(String[] args) {
    String host = args.length > 0 ? args[0] : "127.0.0.1";
    int port = args.length > 1 ? Integer.parseInt(args[1]) :
ComputeServer.DEFAULT_PORT;

    try (DatagramSocket socket = new DatagramSocket()) {
        InetAddress serverAddress = InetAddress.getByName(host);
        send(socket, serverAddress, port, "REGISTER");
        System.out.println("Compute client registered on " + host
+ ":" + port);
    }
}

```

Клиент слушает порт сервера, при получении задачи начинает её выполнение и отправляет результат на сервер.

```

private static void handleTask(DatagramSocket socket, InetAddress
address, int port, String message) throws Exception {
    String[] parts = message.split(";", 5);
    if (parts.length != 5 || !"TASK".equals(parts[0])) {
        return;
    }

    String taskId = parts[1];
    try {
        double lowLimit = Double.parseDouble(parts[2]);
        double highLimit = Double.parseDouble(parts[3]);
        double step = Double.parseDouble(parts[4]);

        RecIntegral recIntegral =
RecIntegral.createCalculationPart(lowLimit, highLimit, step);
        recIntegral.calculateMultiThread();
        send(socket, address, port, "RESULT;" + taskId + ";" +
recIntegral.getResult());
    } catch (Exception e) {
        send(socket, address, port, "ERROR;" + taskId + ";" +
e.getMessage().replace(";", ","));
    }
}

```

RecIntegral (run) #8	RecIntegral (run) #16	RecIntegral (run) #17	RecIntegral (run) #18
run: Сервер	Клиент1	Клиент2	Клиент3
Compute client connected to 127.0.0.1:5000			
Local client port: 49965			
0.02141185963583416			
-0.007427413425016341			
0.00791792255767414			
-0.015447521436032662			
0.027255560929816385			
4.17028567040045E-4			
0.014946696667088255			
-0.023435593555000137			

Результаты полученные 1 клиентом

Рисунок 1 – Результат работы клиента

Вывод: научились создавать клиент-серверные приложения с использованием стандартных классов Java.